

Image Blur, Sharpening & Edge Detection Studio

An interactive image-processing studio powered by a from-scratch 2D discrete convolution engine

PRESENTED BY

Raisul-Bin-Mostofa (2305158) · Mohammad Irfan Tahsin (2305162)

Python

NumPy

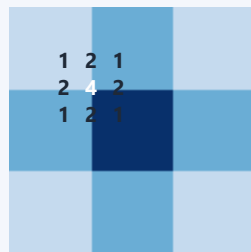
Tkinter / ttk

Pillow

Matplotlib

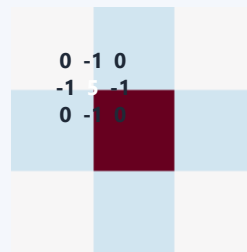
pytest

Gaussian 3×3



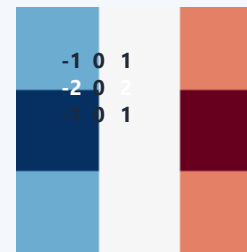
low-pass · $\times 1/16$

Sharpen



high-pass · sum = 1

Sobel G_x



1st derivative · sum = 0

Every filter runs on manual convolution — no scipy / OpenCV / PIL filtering

Why Build an Image Filtering Studio?

- Image filters are everywhere — smartphone cameras, medical imaging, computer vision — but they hide behind one-line calls.
- In Signals & Linear Systems, convolution and kernels stay abstract formulas on the blackboard.
- Calling a black-box filter API teaches the tool, not the mathematics underneath.
- Goal: make the signal processing visible, interactive, and verifiable — in a real working application.

THE PROBLEM WE ADDRESSED

"How can we build a complete desktop application in which every blur, sharpener, edge detector, etc. is implemented from scratch?"

— the question that shaped every design decision

No black-box filters

convolution written by hand

Educational transparency

see the math working live

Mathematical correctness

over visual effects

What We Set Out to Build

1 Manual convolution engine

True 2D discrete convolution: explicit kernel flipping, sliding-window loops, and zero / reflect / edge padding — no library calls.

2 Documented kernel library

Box & Gaussian blur (3–9), basic / strong / Laplacian sharpen, Sobel, Prewitt, Roberts, Laplacian — each with its math documented.

3 Full filter suite

Blur sizes 3×3 – 9×9 with 1–5 iterations, Unsharp Masking, and 4 edge operators $\times G_x / G_y / |G|$ responses with normalization.

4 Tkinter GUI studio

Tabbed workspace: side-by-side comparison, filter chaining, multi-step undo, view modes, and PNG / JPG / BMP import & export.

5 Interactive kernel editor

Editable 3×3 / 5×5 matrices with live preview, numeric validation, preset loading, and kernel metadata (size, sum, min, max).

6 Analysis & verification

Per-channel histograms, image statistics, filter comparison — backed by a full pytest suite with synthetic ground truths.

Signal-First, Layered, and Incremental

1

Engine at the core

One `convolve2d()` implementation powers every filter.
A filter is just: choose kernel → run engine → post-process.

2

Strict layering

Five independent layers. The GUI never computes math;
the signal core never imports Tkinter.

3

Incremental phases

11 phases: scaffolding → engine → kernels → filters →
I/O → GUI → editor → edge controls. `pytest` after each phase.

4

Analytic verification

Synthetic images with known answers: identity kernel,
constant images, impulses, known small matrices.

ONE PIPELINE, EVERYWHERE

Pillow loads the file *PNG / JPG / BMP*



Array → float64 NumPy *signals, not pixels*



Kernel selected *from library or editor*



Manual `convolve2d()` *the engine — our own code*



float64 response kept *negatives preserved*



Clip / normalize *only for display & save*



Show (PhotoImage) or save (uint8) *result delivered*

2D Discrete Convolution — The Heart of the Project

$$(I * K)(y, x) = \sum_j \sum_k I(y - j, x - k) \cdot K(j, k)$$

Input image I (7×7)

94	65	70	89	61	79	84
30	14	36	35	87	91	10
54	83	21	80	20	51	82
36	40	34	74	32	98	49
52	54	61	59	55	98	81
80	72	65	40	98	51	29
85	24	86	64	20	13	49

Kernel K (3×3)

-1	-2	-1
0	0	0
1	2	1

window $\odot K$ flip $\oslash \Sigma$ Output $O = I * K$

.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	-33	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.

highlighted cell: $O(2,2) = \Sigma \text{ window } \odot \text{ flip}(K) = -33$ · the 3×3 window slides over every pixel

SIGNALS VIEW

An image is a 2D discrete signal.
A kernel is an LTI system:

- coefficients sum = 1
→ low-pass (brightness kept)
- coefficients sum = 0
→ high-pass (flat areas → 0)

RGB = three independent
channel signals — the same
kernel runs per channel.

True convolution, not correlation

The kernel is explicitly flipped (`np.flip`) before the sliding window — honouring the mathematical definition.

Padding keeps dimensions

zero · reflect · edge — the strategy decides how borders behave; we default to edge to avoid dark artificial frames.

Odd kernels anchor cleanly

3×3, 5×5, 7×7 ... always have a centre pixel, so the output aligns 1:1 with the input grid.

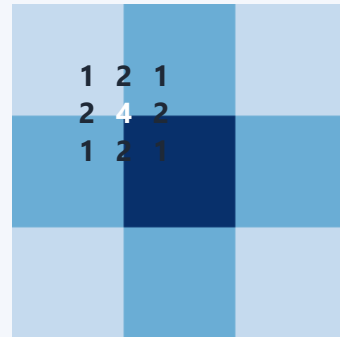
One Engine, Many Kernels

Box blur 3×3



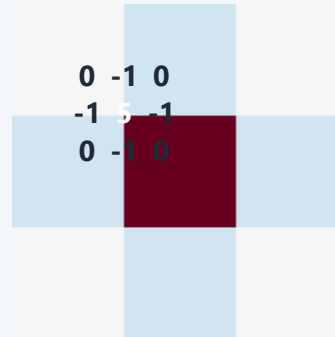
low-pass · averaging · $\times 1/9$ · sum = 1

Gaussian 3×3



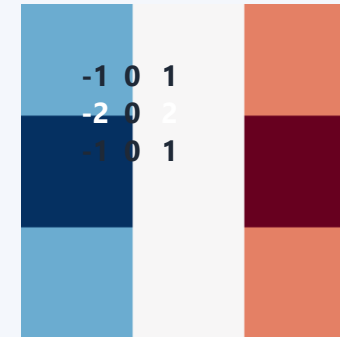
low-pass · weighted · $\times 1/16$ · sum = 1

Sharpen (basic)



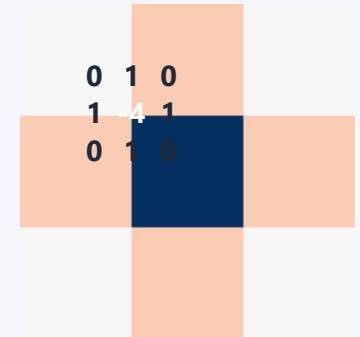
high-pass · centre boost · sum = 1

Sobel G_x



1st derivative $\partial/\partial x$ · sum = 0

Laplacian ∇^2



2nd derivative · isotropic · sum = 0

Unsharp Masking (USM)

$$\text{Out} = I + \alpha (I - G_\sigma * I)$$

sharpen by amplifying the detail mask

Gradient magnitude

$$G = \sqrt{G_x^2 + G_y^2}$$

combines horizontal + vertical edges

Gaussian kernel

$$G(x, y) = e^{-(x^2 + y^2)/2\sigma^2}$$

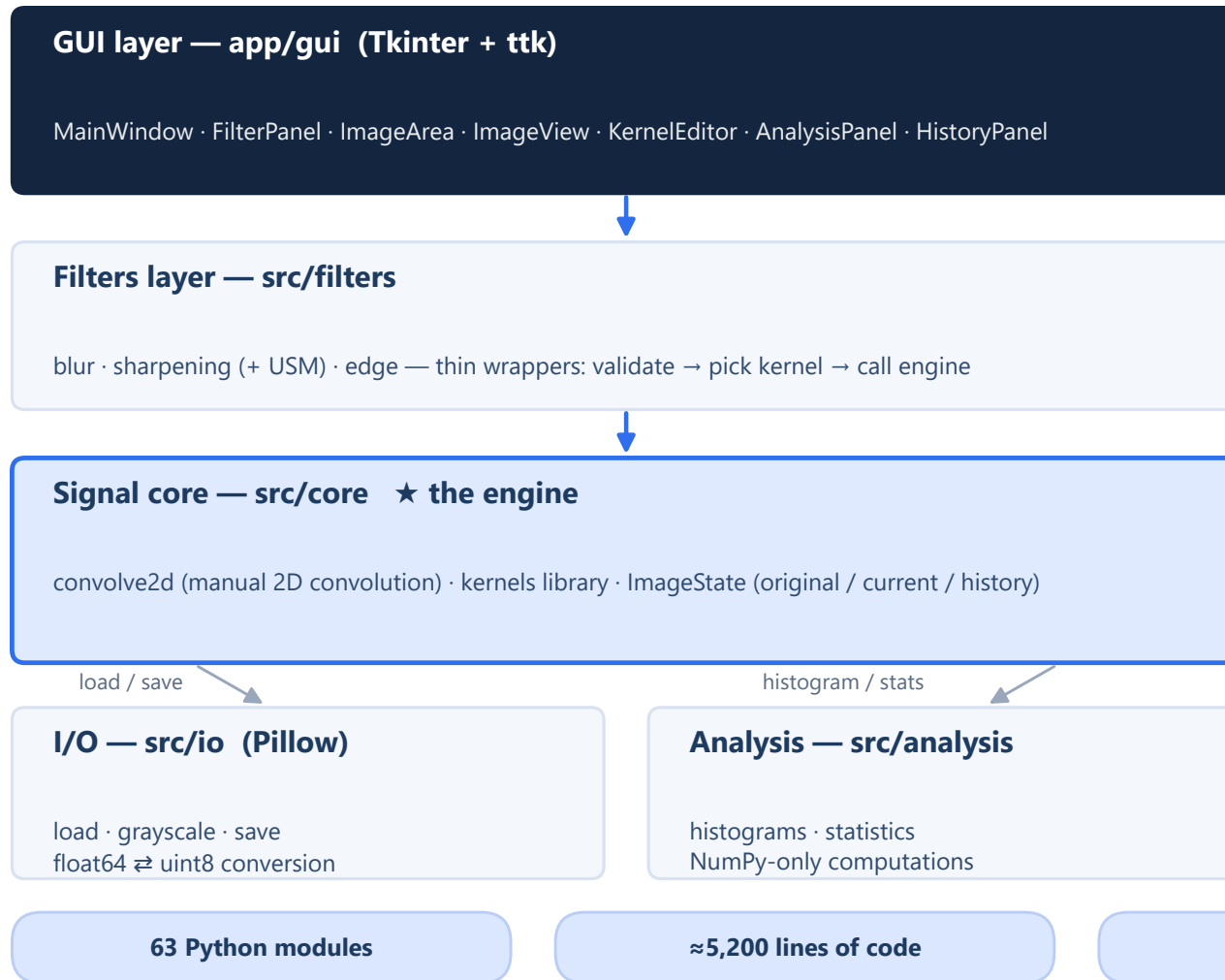
generated at runtime, any odd size

Colour → luma

$$Y = 0.299R + 0.587G + 0.114B$$

ITU-R 601-2 weights before edge ops

Five Strictly Separated Layers



ARCHITECTURAL RULES (ENFORCED)

- ✓ Core imports no Tkinter / no Pillow
- ✓ GUI contains zero convolution math
- ✓ Filters are thin wrappers over the engine
- ✓ ImageState owns original / current / history
- ✓ Every module is independently testable

RESPONSIVE BY DESIGN

Heavy convolution runs in a worker thread; results return to the GUI via `root.after(0, ...)` — the Tkinter event loop never freezes.

Decisions That Shaped the Implementation

Manual loops + explicit np.flip

Honours true convolution (not correlation); NumPy is used only for the inner product and array arithmetic.

float64 end-to-end, clip only at display / save

Edge responses are negative and unbounded — preserving them keeps the mathematics honest before pixels.

Edge padding as the default

Zero padding would paint dark artificial borders around blur / sharpen results; edge replication stays neutral.

Roberts 2×2 embedded in a 3×3

The engine enforces odd kernels, giving every operator a clean centre anchor — one rule, no exceptions.

Gaussian kernels generated at runtime

$G(x,y) = e^{-(x^2+y^2)/2\sigma^2}$ supports any odd size (3–9+), with σ auto-estimated per size.

History stack of pre-filter snapshots

Enables chained filters, multi-step undo, and the Previous Image view — undo-aware by construction.

Worker thread + root.after(0, ...)

Large convolutions run off the main thread; the Tkinter event loop stays live during processing.

Defensive copies of every array

Original, current and history images can never alias or mutate one another — verified explicitly in tests.

A Complete Studio — Not Just Filters

PROCESSING

- 14 filter presets, blur sizes 3×3 – 9×9
- Iterations 1–5 · filters chain on the result
- Multi-step undo · reset · save result
- Previous Image view (undo-aware)

COMPARISON VIEWS

- Side-by-Side · Original · Processed
- Previous · |Original – Processed| difference
- Quick toggle · synchronized display sizing
- Responsive scaling of every view

INTERACTIVE KERNEL EDITOR

- Editable 3×3 / 5×5 coefficient grids
- Live debounced preview on a downscaled copy
- Numeric validation with clear errors
- Metadata: size · sum · min · max

EDGE DETECTION PANEL

- Operator: Sobel · Prewitt · Roberts · Laplacian
- Direction: Gx · Gy · combined |G|
- Response normalization [0, 255]
- RGB auto-converted to luma first

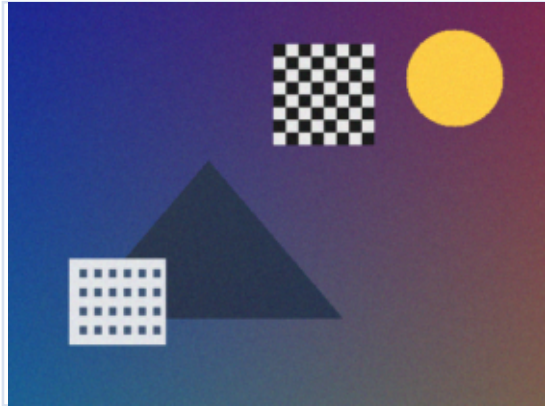
SIGNAL ANALYSIS TAB

- Per-channel 256-bin histograms
- Mean · std · RMS energy · edge density
- Before / after statistics comparison
- Matplotlib plots embedded in Tkinter

MORE WORKSPACE TABS

- Educational kernel info (per filter)
- Multi-filter comparison side by side
- Processing-history chain explorer
- PNG / JPG / BMP load & export

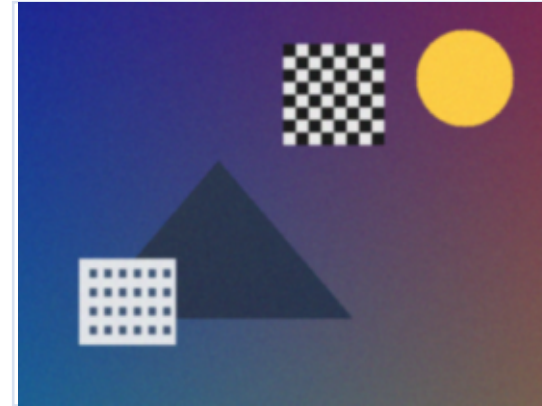
Real Outputs From Our Manual Engine



Original (synthetic scene)



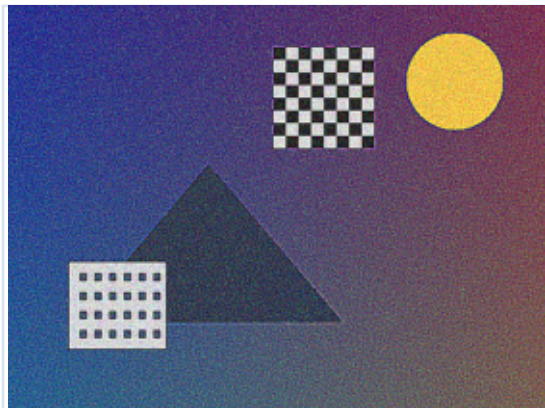
Box Blur 5×5



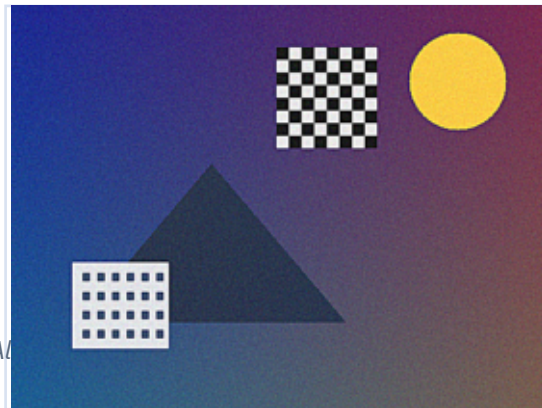
Gaussian Blur 5×5



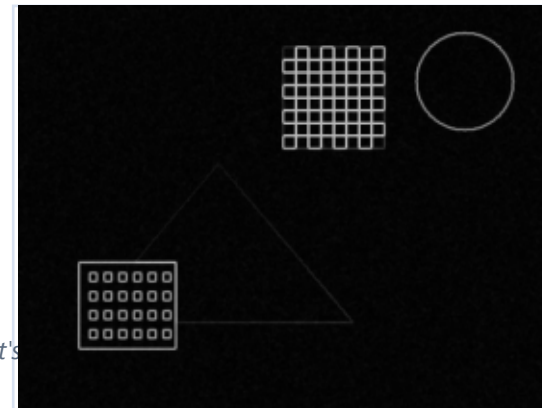
Gaussian Blur 9×9



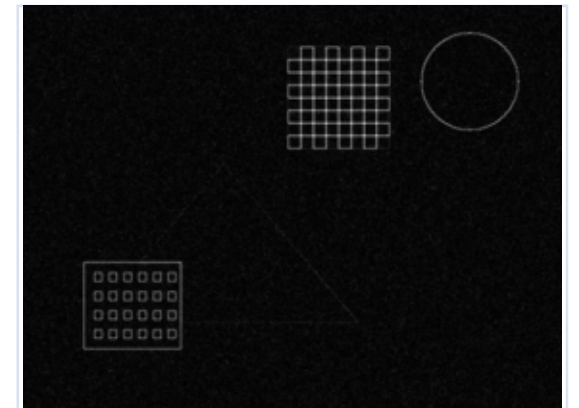
Strong Sharpen
Blur $\Rightarrow$ high-frequency detail removed



Unsharp Mask (r=7, a=1.5)
Sharpen / USM $\Rightarrow$ edges amplified



Sobel $|G|$ (normalized)
Sobel $|G| \Rightarrow$ boundaries appear



Laplacian $|\nabla^2|$ (normalized)
Laplacian $\Rightarrow$ isotropic 2nd derivative

Verified by 130 Automated pytest Tests

130

tests collected

13

test modules

5

coverage areas

0

failures

CONVOLUTION ENGINE

- identity & averaging kernels
- known small matrices, kernel flipping
- zero / reflect / edge padding
- odd sizes 3×3, 5×5, 7×7 + invalid inputs

KERNEL LIBRARY

- dimensions & expected coefficients
- symmetry where applicable
- sum properties (1 for blur, 0 for edges)

FILTERS

- constant / edge / impulse images
- clipping behaviour, blur = low-pass
- Gx vs Gy direction responses, |G| math

IMAGE STATE & WORKFLOWS

- filter chaining, multi-step undo
- reset & new-image session handling
- independent copies — no aliasing
- failed operations never corrupt history

GUI COMPONENTS

- view modes incl. Previous Image
- kernel editor validation & preview
- panel callbacks, end-to-end workflows

ROBUSTNESS

Strict input validation everywhere (TypeError / ValueError with clear messages) · friendly messagebox error reporting in the GUI · defensive copying protects all image state.

Summary & Live Demo Roadmap

WHAT WE DELIVERED

- A complete desktop studio in which every pixel of every filter comes from our own manual convolution.
- A clean five-layer architecture — $\approx 5,200$ lines of Python across 63 modules, 130 tests passing.
- Theory made tangible: kernels, padding, low-pass / high-pass behaviour and gradient operators — live.
- Chaining, undo, Previous Image view, kernel editor, histograms and statistics — all working together.

3-MINUTE LIVE DEMO PLAN

- 0:00** Load an image — side-by-side view
- 0:20** Gaussian blur + chaining, iterations → Previous Image view → Undo
- 1:00** Unsharp Mask before / after + difference view
- 1:30** Sobel edges: G_x vs G_y vs $|G|$, switch operator
- 2:10** Kernel editor: edit coefficients, live preview, apply
- 2:40** Histogram & statistics → save the result as PNG

Thank You — Questions & Discussion

3 min slides · 3 min demo · then Q&A